

Login con Google + persistencia remota (Railway Postgres)

Guía para activar la autenticación con Google y migrar los proyectos y el chat de `localStorage` a **Railway Postgres**. Todo está preparado detrás de un **feature flag** para que la web siga funcionando en modo local hasta que se completen los pasos.

Requisitos previos

1. **Cuenta en Railway** (<https://railway.app>). Plan Hobby es suficiente.
2. **Cuenta de Google Cloud** con acceso al proyecto donde estén configuradas las credenciales OAuth.
3. Acceso al panel de **Vercel** del proyecto `grilliaa` para agregar variables de entorno.

Paso 1 — Provisionar la base de datos

En Railway:

1. Crear un proyecto nuevo → **New Project** → **Deploy PostgreSQL**.
2. En la pestaña **Variables**, copiar el valor de `DATABASE_URL`.
3. Opcional: agregar un servicio de PG Admin o usar `railway connect` desde el CLI para inspeccionar la base.

Paso 2 — Configurar Google OAuth

En Google Cloud Console:

1. **APIs & Services** → **Credentials** → **Create Credentials** → **OAuth Client ID**.
2. Tipo de aplicación: **Web application**.
3. **Authorized JavaScript origins**:
 - `https://grillosia.com`
 - `http://localhost:3000` (para desarrollo)
4. **Authorized redirect URIs**:
 - `https://grillosia.com/api/auth/callback/google`
 - `http://localhost:3000/api/auth/callback/google`
5. Guardar y copiar el **Client ID** y **Client Secret**.

Paso 3 — Variables de entorno en Vercel

En **Settings** → **Environment Variables** del proyecto:

Variable	Valor
NEXT_PUBLIC_ENABLE_AUTH	"true"
NEXTAUTH_URL	https://grillosia.com
NEXTAUTH_SECRET	Cadena aleatoria de 32+ caracteres. Se genera con <code>openssl rand -base64 32</code> .
DATABASE_URL	El valor copiado de Railway.
GOOGLE_CLIENT_ID	Del paso 2.
GOOGLE_CLIENT_SECRET	Del paso 2.

Para desarrollo local, poner los mismos valores en `web/.env.local` (no subir ese archivo — está en `.gitignore`).

Paso 4 — Esquema de la base de datos

El esquema previsto es sencillo. Se puede aplicar con el siguiente SQL desde Railway (pestaña **Query** o vía `psql`):

```

CREATE TABLE IF NOT EXISTS users (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  email       text UNIQUE NOT NULL,
  name        text,
  image       text,
  created_at  timestampz NOT NULL DEFAULT now()
);

CREATE TABLE IF NOT EXISTS projects (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  name        text NOT NULL,
  animal_id   text,
  stage_id    text,
  temp        integer,
  humidity     integer,
  created_at  timestampz NOT NULL DEFAULT now(),
  updated_at  timestampz NOT NULL DEFAULT now()
);

CREATE TABLE IF NOT EXISTS chat_messages (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id  uuid NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  role        text NOT NULL CHECK (role IN ('user','assistant','system')),
  text        text NOT NULL,
  links       jsonb,
  created_at  timestampz NOT NULL DEFAULT now()
);

CREATE INDEX IF NOT EXISTS projects_user_id_idx ON projects (user_id);
CREATE INDEX IF NOT EXISTS chat_messages_project_id_idx ON chat_messages (project_id);

```

Se agregarán también las tablas requeridas por NextAuth (`accounts` , `sessions` , `verification_tokens`) al momento de instalar el adapter de Postgres.

Paso 5 — Activar en la web

Cuando estén los pasos 1-4 hechos:

1. Instalar dependencias adicionales:

```

cd web
pnpm add next-auth@beta @auth/pg-adapter pg
pnpm add -D @types/pg

```

2. Redespregar en Vercel. El flag `NEXT_PUBLIC_ENABLE_AUTH=true` hace que los hooks `useProjects` y `useChat` cambien su fuente de datos a la API con la sesión de NextAuth. La UI **no cambia**.

Comportamiento con el flag apagado (estado actual)

- No se pide login.
- Los proyectos y el chat se guardan en `localStorage` del navegador.
- El botón "Iniciar sesión" no aparece.
- La aplicación funciona sin ninguna variable de entorno obligatoria.

Comportamiento con el flag encendido

- Aparece un botón "Iniciar con Google" en la cabecera para rutas que necesitan sesión (`/proyectos` , `/wizard` , `/chat`).
- Los proyectos se guardan en Postgres y se sincronizan entre dispositivos.
- Los mensajes del chat quedan en Postgres.
- La primera vez que un usuario inicia sesión, sus proyectos locales de `localStorage` se migran automáticamente a su cuenta.

Costos estimados

Servicio	Uso previsto	Costo
Railway Postgres (Hobby)	Base pequeña, tráfico moderado	~\$5 USD/mes tras 30 días de prueba
Google OAuth	Sin límite razonable	Gratis
Vercel (Hobby)	Frontend estático + APIs livianas	Gratis

Rollback

Si algo sale mal después de activar, poner `NEXT_PUBLIC_ENABLE_AUTH=false` en Vercel y redespargar. La web vuelve al modo local sin perder los datos que ya estén en Postgres (siguen ahí para cuando se vuelva a activar).